

Cognizant Digital Nurture 5.0

ReactJS Hands-On Exercise 8

React State

Objective

The objective of this exercise is to understand the concept of **React State** by developing a simple counter application. The application counts the number of people entering and leaving a shopping mall. React State is used to store and update the values dynamically whenever the user clicks the **Login** and **Exit** buttons.

Theory

What is React State?

State is a built-in object in React that stores information about a component. Unlike Props, State can be modified whenever required. Whenever the state changes, React automatically updates the component on the screen.

React State is mainly used for storing data that changes during the execution of the application.

Features of React State

- Stores dynamic data.
 - Local to a component.
 - Mutable in nature.
 - Updated using **setState()**.
 - Automatically updates the UI.
-

Constructor

The constructor initializes the state of the component.

Example:

```
constructor() {
```

```
super();

this.state = {
  entryCount: 0,
  exitCount: 0
};
}
```

setState()

The `setState()` method updates the state whenever the user performs an action.

Example:

```
this.setState({
  entryCount: this.state.entryCount + 1
});
```

Prerequisites

- Node.js
- npm
- Visual Studio Code
- ReactJS

Software Requirements

- React
- JavaScript (ES6)
- HTML
- CSS

Implementation Steps

Step 1

Create a React project.

```
npx create-react-app counterapp
```

Step 2

Move to the project folder.

```
cd counterapp
```

Step 3

Create the following files.

- CountPeople.js
- App.js
- index.js

Step 4

Initialize the state.

Step 5

Create methods to update entry and exit counts.

Step 6

Display the values on the screen.

Step 7

Run the application.

```
npm start
```

Project Structure

counterapp

| — src

| | — CountPeople.js

| | — App.js

| | — index.js

| | — index.css

|

| — public

| — package.json

Source Code

CountPeople.js

```
import React, { Component } from "react";

class CountPeople extends Component {

  constructor() {

    super();

    this.state = {

      entryCount: 0,

      exitCount: 0

    };

  }

  updateEntry = () => {

    this.setState((prevState) => ({

      entryCount: prevState.entryCount + 1
```

```
    });  
  };  
  updateExit = () => {  
    this.setState((prevState) => ({  
      exitCount: prevState.exitCount + 1  
    }));  
  };  
};  
  
render() {  
  return (  
    <div  
      style={{  
        textAlign: "center",  
        marginTop: "80px",  
        fontFamily: "Arial"  
      }}  
    >  
      <h1>Mall Entry Counter</h1>  
      <button  
        onClick={this.updateEntry}  
        style={{  
          padding: "10px 20px",  
          backgroundColor: "green",  
          color: "white",  
          border: "none",  
          marginRight: "10px",  
          cursor: "pointer"  
        }}  
      />  
    </div>  
  );  
}
```

```
    }}  
  >  
    Login  
  </button>  
  <span>  
    {this.state.entryCount} People Entered!!  
  </span>  
  <br /><br />  
  
  <button  
    onClick={this.updateExit}  
    style={{  
      padding: "10px 20px",  
      backgroundColor: "red",  
      color: "white",  
      border: "none",  
      marginRight: "10px",  
      cursor: "pointer"  
    }}  
  >  
    Exit  
  </button>  
  <span>  
    {this.state.exitCount} People Left!!  
  
  </span>
```

```
        </div>
      );
    }
  }
}
export default CountPeople;
```

App.js

```
import React from "react";
import CountPeople from "./CountPeople";
function App() {
  return (
    <div>
      <CountPeople />
    </div>
  );
}
export default App;
```

index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
const root = ReactDOM.createRoot(
  document.getElementById("root")
);
```

```
root.render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
);
```

package.json

```
{  
  "name": "counterapp",  
  "version": "1.0.0",  
  "private": true,  
  "dependencies": {  
    "react": "^18.2.0",  
    "react-dom": "^18.2.0",  
    "react-scripts": "5.0.1"  
  },  
  "scripts": {  
    "start": "react-scripts start",  
    "build": "react-scripts build",  
    "test": "react-scripts test",  
    "eject": "react-scripts eject"  
  }  
}
```

Output Screenshot:



Learning Outcome

After completing this exercise, I learned how to:

- Understand React State.
 - Initialize state using the constructor.
 - Update state using **setState()**.
 - Handle button click events.
 - Automatically update the user interface.
 - Develop interactive React applications.
 - Understand the difference between Props and State.
-

Conclusion

This exercise helped in understanding the practical implementation of **React State** in ReactJS. By creating a counter application, I learned how to initialize state, update it using the **setState()** method, and automatically reflect the changes in the user interface. The exercise also strengthened my understanding of event handling, component lifecycle, and dynamic rendering, making it an essential foundation for building interactive React applications.